

陽明交通大學 114 年度上學期 機器學習 Homework #2

劉益宏 教授 2025/10/13

1. 目的：利用鳶尾花資料(Iris data set)來訓練 Support Vector Machine (SVM)，並分別比較線性與非線性 SVM 所訓練的超平面(hyperplane)有何差異。欲分析的數據集: Iris dataset (150 data points from 3 different classes)
2. 作業內容:

Part1: Linear SVM (Initialization: $C=1$)

- Step1:** 將Iris data set的變色鳶尾(Versicolor, label=2)以及維吉尼亞鳶尾(Virginica, label=3)分別設為positive class與negative class，並採用花瓣長度(3rd feature)與花瓣寬度(4th feature)作為特徵。
- Step2:** 將positive class與negative class的前25筆data設為training data，剩餘的50筆設為test data。
- Step3:** 實現SVM分類器（請勿直接使用開源的SVM套件）
- 補充：在求解對偶問題（Dual problem）時，可以利用python的開源程式(qpsolvers：<https://pypi.org/project/qpsolvers/>)，並請自行將訓練資料矩陣化並參閱前兩者的documentation調整輸入的形式。
- Step4:** 將training data送至SVM中訓練，求解Lagrange multiplier - alpha，再透過Kuhn-Tucker(KT) condition分析並求解bias。(紀錄底線標記之結果，數值請列到小數第四位)。
- Step5:** 將test data輸入至Step4所訓練的SVM decision function，對50筆test data分別進行分類決策，並記錄test data的分類率。
- Step6:** 將penalty weight C分別改為10以及100並重複Step4 - Step5。

```

Linear SVM:
C = 1
alpha =
[1.      0.0667 1.      0.      1.      0.      1.      0.      0.      0.
 0.      0.      0.      1.      0.      0.      0.0667 0.      0.0667 0.
 1.      0.      1.      0.      0.      0.      1.      0.      0.      0.
 0.      1.      0.      0.      0.      1.      0.2    0.      1.      0.
 0.      0.      0.      0.      1.      0.      1.      0.      1.      0. ]
b = [10.54 10.54 10.54 10.54]
Correct Rate = 0.94
CR = 94.00 %

```

```

Linear SVM:
C = 10
alpha =
[ 0.  0.  9.  0.  0.  0. 10.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
 0.  0. 10.  0.  9.  0.  0.  0.  8.  0.  0.  0.  0. 10.  0.  0.  0.
 0.  0.  0.  0.  0.  0.  0.  0. 10.  0.  0.  0. 10.  0.]
b = [15.14 15.14 15.14]
Correct Rate = 0.96
CR = 96.00 %

```

```

Linear SVM:
C = 100
alpha =
[0.000000e+00 3.000000e-04 4.44438e+01 0.000000e+00 4.000000e-04 0.000000e+00
 1.000000e+02 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 1.000000e-04
 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 3.000000e-04 0.000000e+00
 3.000000e-04 0.000000e+00 1.000000e+02 0.000000e+00 4.44438e+01 0.000000e+00
 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 1.000000e-04 0.000000e+00
 0.000000e+00 1.000000e+02 1.000000e-04 1.000000e-04 0.000000e+00 0.000000e+00
 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 2.000000e-04
 0.000000e+00 0.000000e+00 1.000000e+02 0.000000e+00 0.000000e+00 0.000000e+00
 8.88884e+01 0.000000e+00]
b = [11.004 11.0044 11.0041 11.0038 11.004 11.004 11.0044 11.005 11.0056
 11.0052 11.0049 11.0044]
Correct Rate = 0.92
CR = 92.00 %

```

Part2: RBF kernel-based SVM (Initialization: C=10, sigma=5)

Step1 - Step5: 同Part1中Step1 - Step5流程，但採用RBF kernel。

Step6: 將sigma分別改為1、0.5、0.1以及0.05並重複Step4 - Step5。

-----RBF-----

C = 10, sigma = 1

alpha =

```
[0.0000e+00 0.0000e+00 8.8908e+00 0.0000e+00 0.0000e+00 0.0000e+00
 1.0000e+01 3.4600e-01 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
 0.0000e+00 0.0000e+00 1.0000e+01 0.0000e+00 8.8908e+00 0.0000e+00
 0.0000e+00 0.0000e+00 8.1275e+00 0.0000e+00 0.0000e+00 0.0000e+00
 0.0000e+00 1.0000e+01 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
 0.0000e+00 0.0000e+00 1.0000e+01 0.0000e+00 1.0000e-04 0.0000e+00
 1.0000e+01 0.0000e+00]
```

alpha_sum = 76.2552

b = [-0.2193 -0.2193 -0.2193 -0.2193 -0.2172]

CR = 96.00 %

-----RBF-----

C = 10, sigma = 0.5

alpha =

```
[ 0.      0.      5.9852  0.      0.      0.      5.3969  1.0033  0.
 0.      0.      3.006  0.      0.      0.      0.      0.      0.
 0.      0.     10.      0.      5.9852  0.      0.      0.2548  0.
 0.      0.      0.      0.     10.      0.2937  0.      0.1434  0.
 0.      0.      0.      0.      0.      0.      0.      0.5835 10.
 0.      0.1012  0.      10.      0.      ]
```

alpha_sum = 62.7532

b = [-0.2444 -0.2444 -0.2444 -0.2444 -0.2444 -0.2444 -0.2444 -0.2444 -0.2444
-0.2444 -0.2385]

CR = 94.00 %

-----RBF-----

C = 10, sigma = 0.1

alpha =

```
[0.2801 0.3292 1.2411 0.3631 0.0704 0.703 0.7071 1.0221 0.0394 0.867
 1.0131 1.0463 0.2517 0.2801 1.145 0.1056 0.3292 0.9249 0.3292 0.9196
 2.5408 0.3631 1.2411 1.0054 0.9109 0.5211 0.1937 0.7048 0.4102 0.1378
 0.415 1.0583 0.8405 0.7712 0.5245 0.537 0.7111 0.7242 0. 0.7778
 0.7622 0.5161 0.5396 0.7948 2.3432 0.6959 0.6502 0.6137 2.2924 0.4936]
```

alpha_sum = 36.0576

b = [-0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594
-0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594
-0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594
-0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594 -0.1594
-0.1594 -0.1594 -0.1594 -0.1594]

CR = 88.00 %

```

-----RBF-----
C = 10, sigma = 0.05
alpha =
[0.5377 0.3213 0.6269 0.5453 0.9401 0.9632 1.0929 1.1103 0.9401 1.0907
 1.1103 1.1105 0.9579 0.5377 1.1106 1.0552 0.3213 0.9809 0.3213 1.093
 1.2539 0.5453 0.6269 1.093 1.0909 0.7834 0.783 0.8737 0.7831 0.8414
 0.8574 0.8898 0.8894 0.8891 0.7834 0.692 0.889 0.8891 0.6734 0.8893
 0.8893 0.7834 0.8733 0.8893 1.0591 0.8737 0.7977 0.8734 1.0587 0.8731]
alpha_sum = 42.7545
b = [-0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106
-0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106
-0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106
-0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106 -0.1106
-0.1106 -0.1106 -0.1106 -0.1106 -0.1106]
CR = 82.00 %

```

Part3: Polynomial kernel-based SVM (Initialization: C=10, p=1)

Step1 – Step5: 同Part1中Step1 – Step5流程，但採用Polynomial kernel。

Step6: 將 p 分別改為 2、3、4、5 並重複 Step4 – Step5。

```

-----polynomial-----
C = 10, poly = 2

alpha =
[ 0.      0.      5.0666  0.      0.      0.      10.      0.      0.
 0.      0.      0.0474  0.      0.      0.      0.      0.      0.
 0.      0.      10.      0.      5.0666  0.      0.      0.      0.
 0.      0.      0.      0.      10.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.1805  0.      0.      10.
 0.      0.      0.      10.      0.      ]
alpha_sum = 60.361
b = [-0.2103, -0.2103, -0.2103, -0.2103]
CR = 94.00 %

```

```

-----polynomial-----
C = 10, poly = 3
alpha =
[ 0.      0.      4.4255  0.      0.      0.      10.      0.      0.
 0.      0.      0.2591  0.      0.      0.      0.      0.      0.
 0.      0.      10.      0.      4.4255  0.      0.      0.      0.
 0.      0.      0.      0.      10.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      10.
 0.      0.      0.      9.1101  0.      ]
alpha_sum = 58.2203
b = [-0.3462, -0.3462, -0.3462, -0.3462]
CR = 94.00 %

```

-----polynomial-----

C = 10, poly = 4

alpha =

```
[ 0.      0.      4.0198  0.      0.      0.      10.      0.      0.
  0.      0.      0.3418  0.      0.      0.      0.      0.      0.
  0.      0.      10.      0.      4.0198  0.      0.      0.      0.
  0.      0.      0.      0.      10.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      0.      0.      0.      10.
  0.      0.      0.      8.3813  0.      ]
```

alpha_sum = 56.7627

b = [-0.3773, -0.3773, -0.3773, -0.3773]

CR = 94.00 %

-----polynomial-----

C = 10, poly = 5

alpha =

```
[ 0.      0.      3.7789  0.      0.      0.      10.      0.      0.
  0.      0.      0.1575  0.      0.      0.      0.      0.      0.
  0.      0.      10.      0.      3.7789  0.      0.      0.      0.
  0.      0.      0.      0.      9.2312  0.      0.      0.      0.
  0.      0.      0.      0.      0.      0.      0.      0.      10.
  0.      0.      0.      8.484  0.      ]
```

alpha_sum = 55.4303

b = [-0.419, -0.419, -0.419, -0.419, -0.419]

CR = 94.00 %

Part4 : Discussion and results presenting

請試著討論以下問題：

1. Linear SVM與kernel-based SVM所訓練的hyperplane有何差異？
2. 隨著kernel parameter的改變，RBF kernel與polynomial kernel所訓練的hyperplane可能有什麼變化？其與分類率的變化有何關聯？請嘗試解釋之。
3. 設定kernel parameter時，是否有方法避免hyperplane過度擬合(overfitting)的現象發生？若有請詳細討論。

將上列實驗之結果與討論事項整理後做詳細的討論，並以書面報告呈現。

1. Linear SVM 所建立的超平面是一條在原始特徵空間中的直線或平面，因此只適用於資料大致線性可分的情況。它的決策邊界是線性的，數學形式簡單。

kernel-based SVM 會利用 kernel 函數把資料映射到更高維的空間，在高維空間中建立線性超平面，再對應回原始空間後呈現為彎曲或非線性的邊界。因此，kernel-based SVM 能處理非線性可分的資料，並形成較複雜的決策邊界。

2. 在使用 RBF (Gaussian) kernel 時，其中 σ 為 RBF kernel 的參數，控制高斯分佈的寬度。 σ 的大小會直接影響每個資料點對周圍空間的影響力，進而改變 SVM 所生成的超平面的彎曲程度：

- (1) σ 很大時：高斯分佈變寬，每個資料點的影響範圍變大，超平面變得平滑且接近線性模型。此時模型較簡單，容易出現 underfitting，分類率可能偏低。
- (2) σ 很小時：高斯分佈變窄，模型只會專注在各資料點附近，超平面會非常彎曲、緊貼訓練資料，容易出現 overfitting，訓練分類率高，但測試資料的分類率可能下降。
- (3) 適當的 σ 值：能產生既能分開資料又不過於複雜的邊界，此時分類率通常最高且具有泛化能力。

對於 polynomial kernel，其中 d 為多項式次數。當 d 越大時，模型可以擬合更多非線性特徵，決策邊界會變得複雜且彎曲程度增加，但若 d 過高，也會造成 overfitting；反之若 d 太小，模型接近線性。

3. 使用交叉驗證可以有效評估模型是否容易過擬合或具有良好泛化能力。透過調整 kernel 的參數（像是 RBF 的 γ 或 polynomial kernel 的

degree)，並在交叉驗證過程中比較不同參數組合的表現，即可挑選出效果最佳且不易過度擬合的設定。

選對 kernel 非常關鍵，因為不同的 kernel 適合不同的資料特性。

若資料呈現近似線性，可使用 linear kernel；若資料較複雜且非線性明顯，則可以使用 RBF 或 polynomial kernel。然而，kernel 越複雜越容易導致過擬合，因此在選擇 kernel 時，應同時搭配交叉驗證來確認是否合適。

使用網格搜尋 (Grid Search) 可以系統性地找到最佳參數組合。

此方法會逐一測試各種可能的 kernel 參數（例如 C 、 γ 、degree），並依訓練結果選出最表現良好的一組；若不確定參數範圍，也可以先用交叉驗證確認哪些範圍可能產生過擬合，再進一步調整。

控制模型複雜度或減少資料雜訊，也能降低過擬合風險。

例如減少不必要的特徵、使用正規化資料、刪除離群值等方式都能有效提升模型的穩定性。但這些方法是否必要，仍需根據實驗結果決定，而不是盲目套用。